

Introduction:

In today's digital ecosystems, data plays a very important and fundamental role in technical, economic, operational, and strategic success across various industries. Over time, with the growth in the volume, necessity, speed, and types of data, attention to the infrastructure for collecting and storing data and intelligently processing it from all aspects, especially in terms of speed and cost, has gained significant importance. The company **'Wonderland'** (WPG), as an international entity active in the real estate sector, is faces challenges and complexities that growing organizations typically encounter when managing large volumes of diverse and scattered data across different geographical areas. Each branch operates semi-independently and uses old and diverse systems to manage customer affairs. To address this challenge, the company has decided to establish a cloud infrastructure that is capable of collecting data from various sources, secure storage, real-time analytics, and predictive capabilities based on artificial intelligence models. In addition, this company aims to build secure, intelligent, and scalable systems for semi-structured and unstructured data, which are obtained because of collaboration with research institutions Metroshine (**MDAI**).

Company challenges:

- **Various types** of data from different sources with varying **volumes Data**
- **Privacy preservation** and **compliance** with regional regulations such as **GDPR**
- Ability to perform **real-time processing** with large volumes
- **Integration** of purchased data
- Ability to maintain and sustain the **pipeline**
- The need for designing accurate **machine learning models** that can adapt to diverse and sometimes incomplete data
- Designing architectures that can support **predictive models with large volumes** of data.

My Roll:

As a cloud data analyst, my task is to design and implement a data system and pipeline in a cloud environment for WPG. The goal of this infrastructure is to collect and integrate diverse and scattered data such as information from smart homes (Internet of Things), real estate transactions, customer behaviour, and market trends from various sources. Secure data storage and compliance with data privacy regulations, conducting real-time analytics, predicting real estate prices, monitoring unusual behaviours, and providing personalized recommendations are among the objectives of this design. Given the exponential growth of data and the needs of the company, the need for flexibility and scalability is essential, and it should enable advanced analytics and smart decision-making for various teams.

Exploration of Data and Compliance/Technical Requirements

❖ Data Collection Requirements

A combination of **structured**, **semi-structured**, and **unstructured** data that is very effective in achieving an acceptable explanation for system performance and building a highly accurate model. This includes operational data, financial and customer-related data, interaction and sensor data collected from smart homes, websites, and applications, as well as external and research data.

❖ Data Storage Requirements

- Cloud data repository.
- Analytical data warehouse
- Layered Data Management
- High reliability and sustainability
- Backup copies of the data

❖ Analytical Requirements

Based on the analytical needs of the WPG Company, the ability to use advanced technologies based on artificial intelligence and machine learning must be established to conduct rapid, intelligent, and real-time analyses on data obtained from instant transactions and sensors. Additionally, artificial intelligence models should be used to predict property values, classify customers, and provide targeted recommendations. Furthermore, interactive reports on the status of financial profits and losses are provided to managers and stakeholders through dashboards.

❖ Processing Needs

- Real-time ingestion & analytics
- Advanced analytics (AI/ML)

❖ Legal Challenges:

- Compliance with **GDPR** (for Europe), **CCPA** (for the USA)
- Data anonymization for collaboration with **MDAI**

Reason of logical selection of AWS

- Capable of creating a flexible, efficient, and scalable cloud infrastructure for big data and IoT
- Capability of Real-time processing without any physical infrastructure.
- Meets global security standards (encryption, IAM, audit logging)
- Extensive support for GDPR, HIPAA, ISO, SOC, and others
- Rich ecosystem for data gathering, Real-time processing, and ML
- AWS has the greatest variety of services compared to Azure (including IoT, ML, Big Data) and is suitable for real-time analytics and data-driven projects, while being primarily used for Windows and Microsoft-compatible systems.
- AWS enables cost-effective and elastic deployment of services in the cloud (Hoffman and Blok, 2018)

Comparison with Azure and GCP:

One of the reasons for not choosing **Azure** was that it is more focused on **Windows** and **Microsoft products**, which makes it **less flexible** and has less variety in analytical tools and data services. The advantage of **AWS** over

Google Cloud Platform (GCP) was that it is more focused on **Big Query**. Although it has good tools for machine learning, working with them is a bit more complicated. It also has limitations in security services, such as **HIPAA**.

Architecture Design & Diagram on AWS

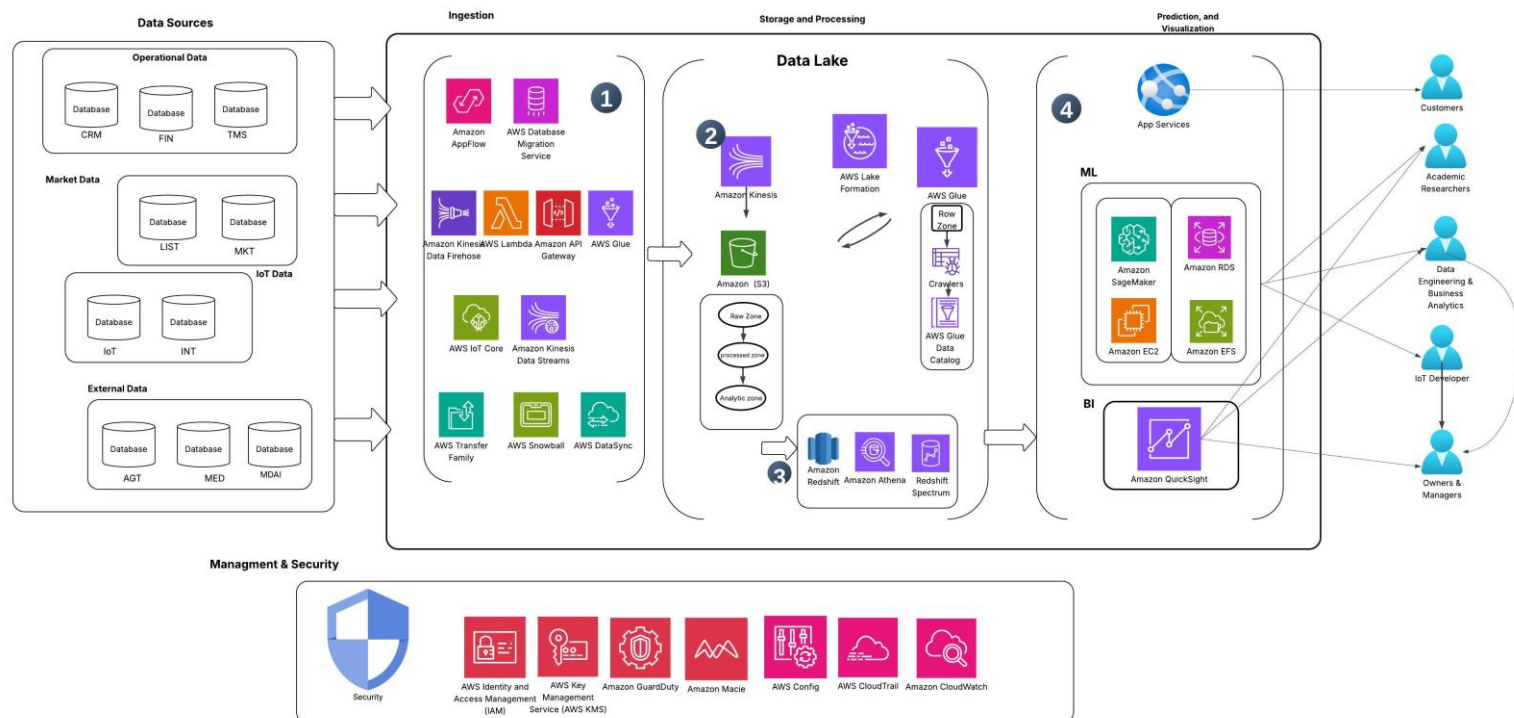


Figure 1- Preview of The Cloud Data Pipeline Diagram Based On AWS

Data Sources

■ Operational Data (structured)

Since this data has structured formats and includes detailed information about customers, their transactions, and contracts, which are stored in the form of rows and columns.

Customer relationship management data (**CRM**): Customer identification, requests and complaints, etc.

Transaction & financial data (**FIN**): Purchase date and amount, invoices and financial receipts, etc.

Customer-related data (**TMS**): Lease agreement between the customer and the owner, lease start and end dates, and repair requests

■ Market & Property Data (structured & semi structured)

Real estate market data such as price, geographic location, property specifications, etc.

Sales and rental information (**LIST**)

Market information and regional price rates (**MKT**)

■ Interaction and sensor data (IoT Data) (structured & semi structured)

Data in formats such as JSON, XML, or signal data that are typically not in tabular form. Such as temperature, humidity, motion, on/off status of devices, etc.

Smart home data (**IoT**)

User interaction behaviour with the website and app (**INT**)

- **External & Research Data** (structured & semi structured & unstructured)

Local and agency data (**AGT**): Registered property listing information, owner or seller information, property viewing hours or real estate agent information

Research data (**MDAI**): Anonymous data related to tenant behavior, urban infrastructure data such as population, traffic, and research results such as property price predictions

Multimedia content such as images and videos (**MED**): Exterior photos of the building, virtual tour video, and architectural or interior design floor plans

Data Ingestion

At this stage, we read data from various sources and enter it into a central database.

- To transfer **operational data** using a tools, such as **Amazon Appflow** that uses a graphical interface to directly transfer data from **SaaS** sources or external applications to **AWS**, and **AWS DMS** is for simultaneous data transfer between database resources. It should be noted that transferring resources from **Amazon S3** to databases requires separate processing (**Glue Jobs**).

SaaS (Software as a Service): It is a cloud-based model that provides access to ready-made applications over the Internet without the need to install, maintain, or update software.

- For transferring **Market Data**, We use tools, such as **AWS Lambda**, **API Gateway** for extracting data from ¹**APIs**, and collecting files from **external sources**, and **Amazon Kinesis Data Streams/Firehose** is used for gathering **real-time** data, (Marz and Warren, 2015)

Note: One of the reasons for mentioning the **AWS Glue** tool in the **Ingestion** section is that the raw data is immediately implemented by **ETL** operations after entering **S3**, or the data structure is determined with the **Catalog service**. Which is the first step to enter the analysis phase. In many architectures, **ETL** is displayed simultaneously with **Ingestion**.

- To access **real-time data** and **sensors**, tools, such as **AWS IoT Core** for direct reception from **IoT sources** like smart lock or temperature sensors **Amazon Kinesis Data Streams** for accessing real-time and instantaneous user information like their clicks **Amazon MSK** for managing the reception and distribution of real-time data **AWS Lambda** for situations where a quick action is needed when uploading a file.

¹ Application Programming Interface

- To obtain data such as images, videos, urban data, we use tools, such as **Aws Transfer Family** to receive files from local agencies or colleagues, **AWS Snowball** to receive large volumes of physical data, and **AWS DataSync** for transferring files from local servers

Note: After the data transfer, ultimately all data is stored in **Amazon S3** (Raw, Processed, Analytics Zones), then it enters the stages of processing, modelling, and data visualization.

Storage and Processing

Data Lake: In this project, to increase scalability for the company's future development in various geographical areas and to perform real-time processing, analysis, machine learning, and dash boarding, as well as apply policy for access control, we use of **Data Lake** that is able to apply on various type of data.

Initially, the data is collected from sources using the tools introduced in section **Ingestion**. Streaming (real-time) data are directly entered into **Amazon Kinesis** upon arrival and then transferred to **Amazon S3** for any analytical

perations, **ETL**, or **SQL** based processing. Other types of data (CSV, JSON, Parquet, ORC), and encrypted data, are entered to **Amazon S3** as **segregated buckets**. Once the data comes in, it is passed to other parts based on the type of process we want to apply to it. It is worth noting that in **Amazon S3**, no processing is done on the data and the data is completely raw. **Raw data** is transferred from **Amazon S3** to three selected paths: **Data Glue**, **Amazon Lake Formation**, or **Analyse Layer**.

Internal structure Amazon S3: First, in the **Raw Zone** all the raw data is stored without any changes. (CSV, IoT events and JSON), Then in the **Processed Zone** (or Silver Layer), the raw data is pre-processed such as cleaning, normalizing and converting to usable formats and handling missing values, which helps in improving query speed. In the **Analytics Zone** stage, the final and useful data that is suitable for analysis, reporting, visualization and modeling is filtered.

The relationship between **AWS Glue** and **Amazon S3** is bidirectional. This means that **AWS Glue** can read raw data from **Amazon S3** and transforms it into processed data through the **ETL** process, using **Crawlers** to examine the files and extract information such as file format, what fields of what type exist, and stores them in the form of **metadata** tables in **AWS Data Catalog**. Ultimately, it is either returned to **Amazon s3** through **Glue Jobs** tool, or sent to analytical tools like **Athena**, **Redshift**, **ML**, and **visualization** layers.

Amazon Lake Formation creates a space called **Data Lake Location** in **S3** and applies a high level of access control over databases through the Lake Formation Permissions service. Then, by using **Crawler** or **Glue Job**, it registers the

schema, and we can query the data using analytical tools such as **Athena** or **BI** tools, without any risk of information leakage. This service is recommended for pipelines that involve a very high volume of data or are accessed by Multiple user teams, as it has capabilities for precise access control, data cataloging, security, and search. The next step after ingestion and storage & processing layers is the analytical and Machine Learning layers. (Amazon Web Services, Inc., 2020).

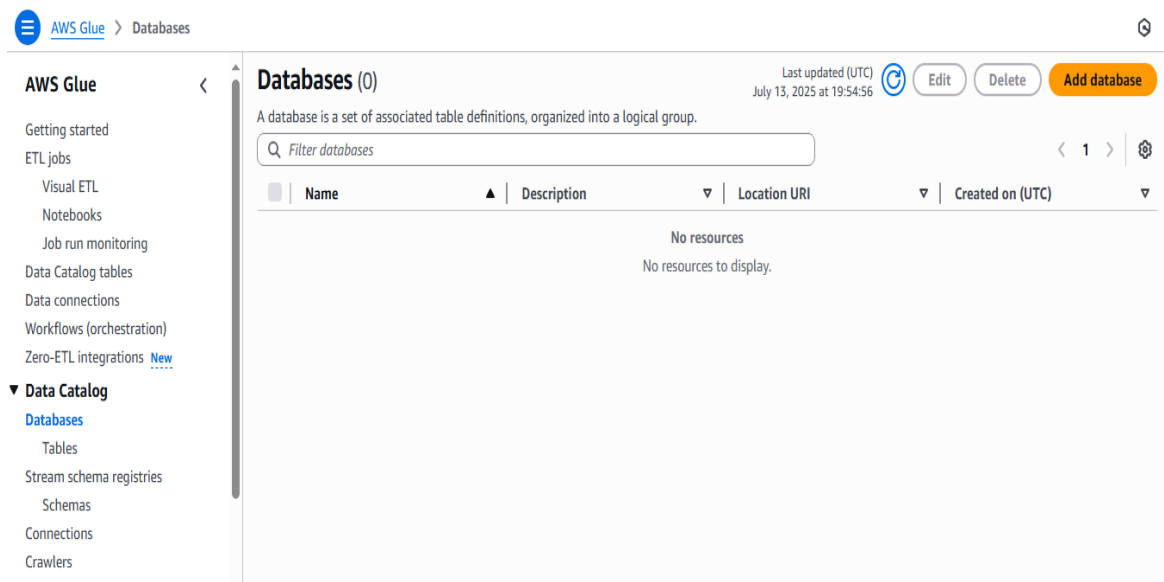


Figure 2- Preview of AWS Glue Service in AWS Console

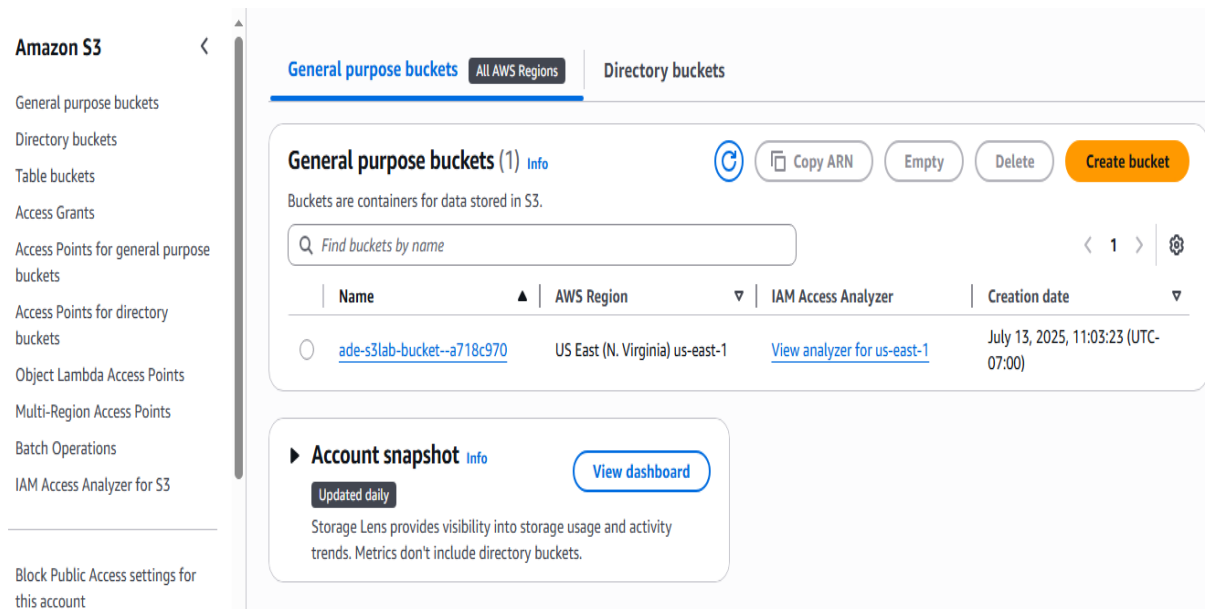


Figure 3- Preview of Amazon S3 Service in AWS Console

Analytic Layer

At this stage, if we want to directly query the data, **Athena(server less SQL engine)** is recommended. There is no need to transfer data or perform **ETL** on the data, and it supports semi-structured data. For example, this tool is useful for simple and quick analyses, such as real estate prices and for quickly analysing customer data or rapidly searching for suspicious records. About **Redshift**, It is an analytical data warehouse where data is transferred to an internal space for analysis. It supports more structured data and is suitable for heavy and complex analyses on large volumes of data. After **ETL** the data enters Redshift in a cleaned form. In this project, it is used for reports such as the average house prices in various neighborhoods over the past 5 years. If I want to apply the same heavy analyses on the data in **Amazon S3**, we use **Redshift Spectrum**. In fact, this service is a feature in Amazon Redshift that allows us not to upload the data into Redshift.

One of the goals of the agency is to forecast market trends using artificial intelligence models. Using the **Segemaker** models are created to predict property prices and cluster them based on area, region, and other features. To store

Structured information such as property details (address, year of built, price, and area) or customer data, transactions, and visits, we use the tool **Amazon RDS**, which allows connection to the BI dashboard for analyzing and reporting of Transactions. **Amazon EC2** creates an environment customized for analysts, allowing the execution of scripts based on an **API**, for instance, price prediction based on **Segamaker** models. **Amazon EFS** shares raw processed files and data among analysis teams, BI, etc. It stores trained models and generates reports with the extensions **CSV/Excel**. **ECD** It connects to **Segemaker** simultaneously to create integration.

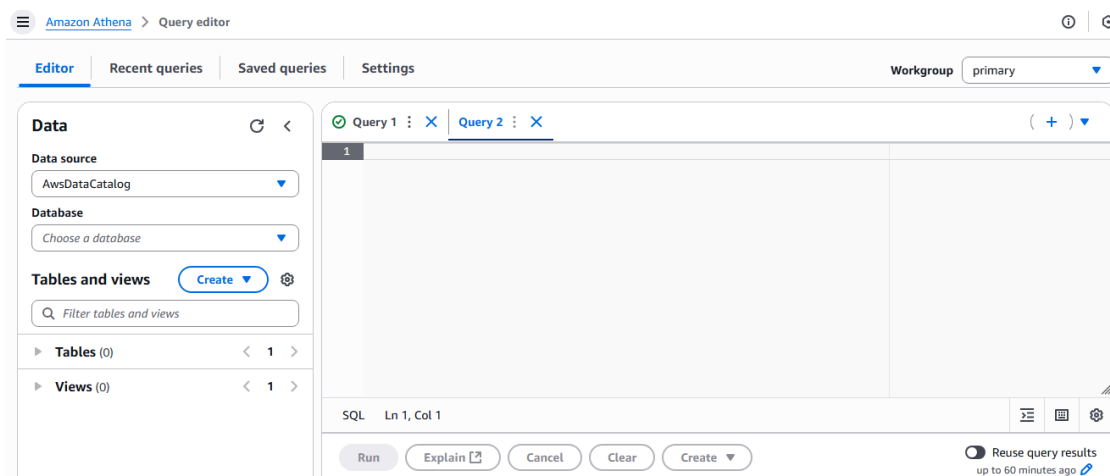


Figure4- Preview of Athena Service in AWS Console

Users groups in Wonderland

- One of the user groups of the agency are **customers**. One of their communication methods with the agency for buying, selling, or getting information about price forecasts are **Customer Applications**, that includes a collection of **dashboard outputs** to display fluctuations, **artificial intelligence models**, **searches**, and so on.
- **Academic researchers** who survey interactive data between agencies and research centers to analyze the real estate market models. This group of users has access to **AI model reports** and **Amazon Quicksight dashboards**.
- **Data Engineering & Business Analytics** For making business decisions, regularly reviewing and comparing predictive models and trying to improve their performance by examining the **models**, **dashboards**, and **Managers**.
- **IoT Developer** integrates and manages Internet-connected devices (such as sensors, cameras, smart equipment) and processes or analyzes their data in the cloud infrastructure.
- **Owners & Managers** make executive decisions based on reports obtained from analytical dashboards.

Security layer

To manage user and service access (who has access and to what level) to AWS resources is one of **IAM** tool responsibilities. The **AWS KMS** tool is used for managing and encrypting data and sensitive information. **Amazon GuardDuty** is used to detect suspicious behaviors or hacking attacks. **Amazon Macie** protects sensitive data such as personal and banking information. **AWS Config** checks changes in accordance with security compliance rules. **AWS CloudTrail** records all activities of users and services. **Amazon CloudWatch** monitors the performance of systems and resources (like CPU usage, memory, traffic, and automatic alerting).

Cost Efficiency Analysis:

In the implementation of cloud, the removal of physical and hardware infrastructure, optimization of resource utilization, and standard automation (**GDPR**) significantly affect the reduction of **operational** and **labour costs**. According to the architecture designed for the **Wonderland** real estate group, the deployment of this cloud structure based on **AWS** will reduce the total cost of ownership (**TCO**) over a **10-year** horizon. In the **On-Premise** infrastructure, elements such as the need for servers, energy, licenses, and maintenance operations lead to

estimated costs between **1,800,000** and **2,000,000** euros. After migrating to the cloud-based infrastructure (**AWS**), due to scalability, the use of **Glue** and **Lambda** services for cost-effective processing of high-volume transactions and the highly affordable tools **Power BI** and **Quicksight** has reduced costs to **1,300,000** to **1,500,000** euros, It indicates a reduction of **%43** percent, which on average amounts to **780,000** Euros.

Unit	Estimated saving
Storage	120000 € (Amazon S3)
ETL Process	90000 € (Aws Glue)
Real – time processing	90000 € (Kinesis + Lambda)
Analysis Intelligence	100,000 €(QuickSight , Athena ,Redshift)
ML	120,000 € (SageMaker)
Security	60,000 € (IAM, Macie, KMS, CloudTrail)
Maintenance and human resources	200000 €

Implementation

The implementation is based on **Python** code, **Boto3** library, and **YAML** language in **AWS Cloud Formation**.

Objective

Our goal is to implement a pipeline for collecting and simulating IoT data, converting it into **real-time data (AWS Kinesis or Firehose)**, transferring it to **S3**, and performing the **ETL** process (**AWS Glue**). Finally, analysis and visualization will be done with one of the two options **Quicksight** or **Segamaker**, which **Quicksight** does not require coding and it is only necessary to connect **S3**.

- IoT Sensors
- Data Ingestion via Amazon Kinesis Firehose
- Data Storage in S3 Data Lake
- Data Transformation via AWS Glue
- Machine Learning Model via SageMaker
- Visualization via Amazon QuickSight

❖ IoT Data Generator

```
import boto3
import json
import random
import time
from datetime import datetime

firehose = boto3.client('firehose', region_name='eu-central-1') # Set it according to your region
delivery_stream_name = 'iot-delivery-stream' # The name of the Firehose you created

def generate_sensor_data():
    return {
        'sensor_id': f'sensor-{random.randint(1,5)}',
        'timestamp': datetime.utcnow().isoformat(),
        'temperature': round(random.uniform(18.0, 30.0), 2),
        'humidity': round(random.uniform(30.0, 60.0), 2),
        'motion': random.choice([0, 1])
    }

while True:
    record = generate_sensor_data()
    response = firehose.put_record(
        DeliveryStreamName=delivery_stream_name,
        Record={'Data': json.dumps(record) + '\n'}
    )
    print(f"Sent: {record}")
    time.sleep(1) # Sending once every second
```

Figure5- Preview of python codes for Generating IoT Data

This code generates random data such as motion status, humidity, temperature, and similar types of data. Every second, a record, **JSON**, is sent to an object of **Firehose Stream**. **Firehose** is responsible for receiving input data and sending it to a specified destination, such as **S3**. The statement **time. Sleep (1)** indicates that the process will repeat every second, thus simulating a real data stream. Therefore, this code represents a part of your smart home system where sensors continuously generate data and this data is sent via **Kinesis Firehose**.

Create data stream [info](#)

Data stream configuration

Data stream name

Acceptable characters are uppercase and lowercase letters, numbers, underscores, hyphens and periods.

Data stream capacity [info](#)

Capacity mode

☒ **On-demand**
Use this mode when your data stream's throughput requirements are unpredictable and variable. With on-demand mode, your data stream's capacity scales automatically.

☐ **Provisioned**
Use provisioned mode when you can reliably estimate throughput requirements of your data stream. With provisioned mode, your data stream's capacity is fixed.

On-Demand mode dynamically scales Amazon Kinesis Data Streams based on your throughput needs, with pay-per-use pricing. For details on limits, visit [Quotas and Limits](#). For details on pricing, visit [Kinesis pricing for on-demand mode](#).

Figure6- Preview of Amazon Kinesis in AWS Console

❖ Kinesis/Firehose Configuration Instructions

```
import sys
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from awsglue.dynamicframe import DynamicFrame

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

# Reading from S3
input_path = "s3://iot-smart-pipeline/raw/"
output_path = "s3://iot-smart-pipeline/processed/"

df = spark.read.json(input_path)

#Remove incomplete records and create a datetime field
df_cleaned = df.na.drop()
df_cleaned = df_cleaned.withColumn("event_hour", df_cleaned["timestamp"].substr(12, 2))

#Save in parquet format
df_cleaned.write.mode("overwrite").parquet(output_path)
```

Figure7- Preview of python codes for Kinesis Configuration

It reads the **raw JSON** generated codes from **S3**, performs some simple operations like deleting incomplete records and extracting event times from the **timestamp** column. Then it saves parquet files of the cleaned data back to **S3**. This process is for **optimizing** and **preparing** the data for useful analyses.

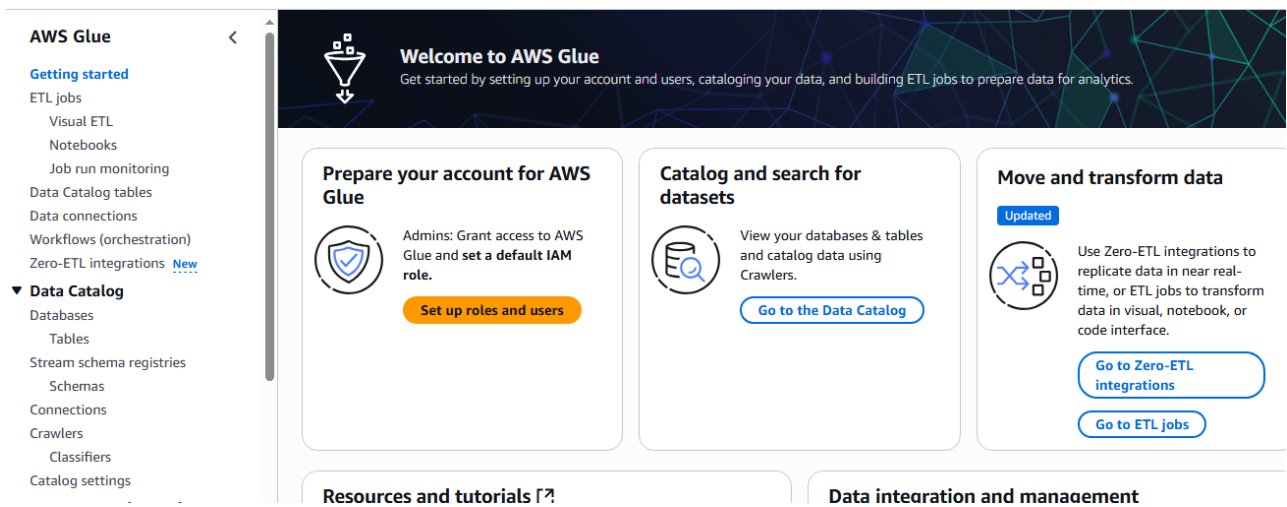


Figure8- Preview of AWS Glue in AWS Console

❖ Using Terraform to build Firehose:

The goal of this code is to implement a simple, secure, and automated **pipeline** for collecting and storing input data from **IoT** devices (sensors) in **AWS**. Ultimately, after defining a **Kinesis Firehose** stream, the data is transferred to **S3** and utilizes an **IAM security** layer for access.

Note: As previously mentioned, **QuickSight** does not require coding because it is a web-based BI service. The processed data generated by Glue and stored in S3 connects to **QuickSight**, which builds dashboards with a UI (similar to loading in Tableau).

Amazon S3 > Buckets > Create bucket

Create bucket [Info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type [Info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket name [Info](#)

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn More](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Figure9- Preview of Creating Bucket in Amazon S3

❖ ML model in SageMaker

Building predictive models using SageMaker allows for automated resource provisioning and API deployment for real-time inference (Sallam, Liguori and Stein, 2021)

```
import pandas as pd
import boto3
import sagemaker
from sagemaker.sklearn import SKLearn
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import joblib
import os

# Parameters
bucket = 'iot-smart-pipeline'
prefix = 'processed/'
region = 'eu-central-1'

# Load data from S3
s3_client = boto3.client('s3')
s3_path = f's3://{bucket}/{prefix}iot_data_transformed.csv'

df = pd.read_csv(s3_path)
df.dropna(inplace=True)

# EDA
print(df.describe())

# Train/Test Split
X = df[['humidity', 'motion']] # features
y = df['temperature']         # target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```

# Train model
model = LinearRegression()
model.fit(X_train, y_train)

# Save model
os.makedirs('model', exist_ok=True)
joblib.dump(model, 'model/model.joblib')

# Upload model to S3
s3 = boto3.resource('s3')
s3.Bucket(bucket).upload_file('model/model.joblib', 'models/model.joblib')

# Deploy to SageMaker
role = sagemaker.get_execution_role()
sklearn_estimator = SKLearn(entry_point='inference.py',
                             role=role,
                             framework_version='0.23-1',
                             instance_type='ml.t2.medium',
                             source_dir='.',
                             dependencies=[])

predictor = sklearn_estimator.fit({'train': s3_path})

# inference.py

```

```

# inference.py

import joblib
import os
import numpy as np

def model_fn(model_dir):
    model = joblib.load(os.path.join(model_dir, "model.joblib"))
    return model

def predict_fn(input_data, model):
    prediction = model.predict(input_data)
    return prediction

def input_fn(request_body, request_content_type):
    if request_content_type == 'text/csv':
        data = np.fromstring(request_body, sep=',').reshape(1, -1)
        return data
    raise ValueError("Unsupported content type")

def output_fn(prediction, accept):
    if accept == 'application/json':
        return str(prediction.tolist()), accept
    raise ValueError("Unsupported accept type")

```

Figure10- Preview of python codes for Modelling

In this piece of code, the data is first read from **S3**. After pre-processing and preparing the data for training, the model is created and transferred to **S3** as a file. Then it is moved from **S3** to **SageMaker**, where the steps of model creation, training, and testing are carried out, thus enabling the prediction of the real estate market.

Note: Any other system such as mobile apps can access the model deployed on **SageMaker** without any infrastructure requirements. **SageMaker** automatically sets up a server and loads the model onto it. Then it creates an **API endpoint** that other systems can send simple requests to and receive **real-time predictions**.



Figure11- Preview of Amazon SageMaker

Cloud Formation YAML Template:

Description: Real Estate Data Pipeline (Kinesis Firehose to S3 with Glue Job)

Parameters:

BucketName:

Type: String

Description: Name for the S3 bucket

Resources:

RealEstateBucket:

Type: AWS::S3::Bucket

Properties:

BucketName: !Ref BucketName

VersioningConfiguration:

Status: Enabled

FirehoseRole:

Type: AWS::IAM::Role

Properties:

RoleName: firehose-s3-role

AssumeRolePolicyDocument:

Version: '2012-10-17'

Statement:

- Effect: Allow

Principal:

Service: firehose.amazonaws.com

Action: sts:AssumeRole

Policies:

- PolicyName: firehose-s3-access

PolicyDocument:

Version: '2012-10-17'

Statement:

- Effect: Allow

Action:

- s3:PutObject

- s3:PutObjectAcl

Resource: !Sub "\${RealEstateBucket.Arn}/*"

FirehoseDeliveryStream:

Type: AWS::KinesisFirehose::DeliveryStream

Properties:

DeliveryStreamName: RealEstateDataStream

DeliveryStreamType: DirectPut

S3DestinationConfiguration:

BucketARN: !GetAtt RealEstateBucket.Arn

RoleARN: !GetAtt FirehoseRole.Arn

Prefix: "incoming/"

CompressionFormat: UNCOMPRESSED

4 GlueDatabase:

Type: AWS::Glue::Database

Properties:

CatalogId: !Ref AWS::AccountId

DatabaseInput:

Name: real_estate_db

GlueRole:

Type: AWS::IAM::Role

Properties:

RoleName: glue-etl-role

AssumeRolePolicyDocument:

Version: '2012-10-17'

Statement:

- Effect: Allow

Principal:

Service: glue.amazonaws.com

Action: sts:AssumeRole

ManagedPolicyArns:

- arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole

GlueJob:

Type: AWS::Glue::Job

Properties:

Name: CleanRealEstateData

Role: !GetAtt GlueRole Arn

Command:

Name: glueetl

ScriptLocation: !Sub "s3://\${BucketName}/scripts/real_estate_etl.py"

PythonVersion: "3"

DefaultArguments:

"--job-language": "python"

"--TempDir": !Sub "s3://\${BucketName}/temp/"

GlueVersion: "3.0"

MaxRetries: 1

Timeout: 10

NumberOfWorkers: 5

WorkerType: Standard

Outputs:

S3BucketName:

Value: !Ref RealEstateBucket

Description: S3 bucket for storing data

FirehoseStreamName:

Value: !Ref FirehoseDeliveryStream

Description: Kinesis Firehose Stream Name

▲ **GlueJobName:**

Value: !Ref GlueJob

Description: Name of the AWS Glue ETL Job

Example of implementation a Temperature Spike Detection and Notification Workflow

Let us consider a situation where sensor data indicates a sudden increase in temperature. The goal is to send a report to administrators or store it in a database. The temperature sensor sends a message over the Internet to IoT Core. Then the rule in AWS IoT Rule Engine is triggered, the same rule that we defined in IoT Core. If the temperature is greater than 70, send this message to another service. In the next step, the above message is sent to AWS Lambda. Lambda processes the message, creates a report, and sends it to the intended user.

Suggestions for the future expansion of Wonderland Property Group's infrastructure:

- Strengthening monitoring and real-time tracking of service statuses and pipelines. It is recommended to use **Amazon SNS** for automatic notifications in case of errors.
- Implementing advanced security infrastructures that ensure data security and compliance with legal requirements: **Encryption at various levels** (S3 encryption, KMS, Glue job encryption) , **IAM Roles** with Principle of Least Privilege, **Audit logs** with AWS CloudTrail Access logging on resources like **S3**.
- Pipeline enhancement to support multiple owners and tenants (Multi-Tenant) and regional control (Region-Aware Processing).
- Integrating more resources.
- Advancement, training, and continuous testing of **ML models**

Conclusion:

In this project, we decided to implement a cloud infrastructure for a company operating internationally in the real estate sector (**Wunderland Real Estate Group**). The cloud infrastructure is based on AWS, capable of simultaneously receiving and processing data in real time (**Kinesis** or **AWS IoT Core**), data preparation (AWS Glue), storage (Amazon S3), analysis and modelling (**SageMaker**), and visualization (**QuickSight**). This design offers the benefit of scalability while maintaining security and compliance with data protection regulations (such as **GDPR**), along with a cost reduction of **25** to **30** percent compared to physical infrastructures. First, we designed a pipeline diagram and considered all aspects, and then we created and tested the pipeline by implementing a simple prototype of this infrastructure using hypothetical data. Due to the rapid reception of data and its storage and analysis in the shortest time possible, we received good performance from the system.

References:

- Hoffman, B. and Blok, M. (2018) Amazon Web Services in Action. 2nd ed. Shelter Island, NY: Manning Publications.
- Amazon Web Services, Inc. (2020) AWS Storage and Data Transfer Services Overview. Seattle, WA: Amazon
- Sallam, A., Liguori, F. and Stein, D. (2021) Machine Learning Engineering on AWS: Using SageMaker and the AWS AI and ML Ecosystem. Sebastopol, CA: O'Reilly Media.